Modelling Steam Game Success Tiers from Pre-Release Metadata: An Ordinal Regression Approach

Zawad Ahsan
Department of Computer Science and Engineering
BRAC University
Dhaka, Bangladesh
23201136
zawad.ahsan@g.bracu.ac.bd

Md. Nurul Asif
Department of Computer Science and Engineering
BRAC University
Dhaka, Bangladesh
23201489
md.nurul.asif@g.bracu.ac.bd

Yeamin Alam
Department of Computer Science and Engineering
BRAC University
Dhaka, Bangladesh
23241007
yeamin.alam@g.bracu.ac.bd

***Abstract*—Predicting a video game's success before launch helps developers and publishers plan budgets, marketing, and platform support. Most prior work on Steam game popularity needs post-launch signals such as player counts and reviews. Other prior work treats success as an unordered class label. Both choices lose information: post-launch signals cannot support pre-launch decisions, and unordered labels ignore the natural order between low and high success. This paper treats success-tier prediction as an ordinal regression problem built entirely from pre-release metadata. A cleaned and leakage-checked dataset of 101,276 Steam titles is built from 125,855 raw records, with 54 engineered features covering pricing, platform reach, store page content, and studio history. The target is a four-tier ordinal label, Niche, Moderate, High, and Hit, built from estimated owner counts. Five baseline classifiers are compared on a stratified 80/20 split, along with hyperparameter tuning, class weighting, an ordinal reframing of a boosted regression output, and a LightGBM model. The ordinal regression model gives the best result, with a macro F1 score of 0.5594 and a quadratic weighted kappa of 0.6691. Its errors are almost always within one tier of the true label. A leakage check supports this result: macro F1 stays at 0.5242 after removing the model's two most dominant features. The paper also examines severe class imbalance, where the Niche tier accounts for 74.8 percent of titles, and states the limitations of hand-set tier thresholds and a single random seed.**

***Index Terms*—class imbalance, game analytics, machine learning, ordinal regression, pre-release prediction, Steam platform**

I. Introduction

The video game industry releases thousands of new titles every year on digital storefronts. Steam is the largest of these storefronts for personal computer games [1]. Most of these titles never reach a wide audience. A small number of releases draw most of the attention.

This gap between the number of games released and the number of games that succeed creates a real planning problem for developers and publishers. A studio can estimate how a game is likely to perform before release. This estimate can guide budget allocation, marketing spend, and platform strategy. Revenue, player counts, and reviews are only known after a game has already launched. Any prediction meant to help pre-launch decisions must use information available before that point.

Existing work on Steam game success falls into two groups. The first group predicts popularity using signals that only exist after release. Examples include early player counts [1], review sentiment [13], and a mix of Steam, Twitch, and Metacritic engagement [9]. These models are accurate. They cannot be used before a game has launched, because the input data does not exist yet.

The second group restricts itself to metadata that is known before release, such as category, supported language count, and developer profile [4]. Almost all of this work treats the outcome as a binary or unordered class label. A model that predicts popular or not popular throws away information about how much a game succeeded by. In practice, success is graded. A niche title, a moderate seller, and a breakout hit are not equally different from each other. An unordered classifier is not built to reflect that.

This creates the research gap this paper addresses. Limited published work treats pre-release success prediction as an ordinal problem. In an ordinal problem, the tiers of success have a defined order. A model can then be scored on how close its errors are to the true tier, not only on whether it is exactly right. This paper closes that gap. It uses a large, cleaned, and leakage-checked Steam dataset built entirely from metadata

that a developer would have in hand before a game goes live.

The contributions of this paper are:

- A four-tier ordinal success target built from estimated owner counts, together with an explicit leakage guard that keeps only features known before release.
- A pipeline of 54 engineered features covering pricing, platform reach, store page content, language support, and studio history, built from a raw Steam metadata dump of 125,855 titles.
- A comparison of five baseline classifiers against a tuned gradient boosting model, a class-weighted variant, an ordinal regression reframing, and a LightGBM model, all evaluated with metrics suited to an imbalanced, ordered target.
- A leakage check and a permutation importance analysis that identify which pre-release features drive the prediction and confirm the result does not depend entirely on the two most dominant features.

Section II reviews related work on game and product success prediction, grouped by theme. Section III describes the data pipeline and the ordinal regression method. Section IV describes the experimental setup, including the dataset, metrics, and implementation details. Section V presents the results and discusses why the ordinal model outperforms the alternatives. Section VI states the limitations of the study. Section VII concludes with directions for future work.

II. Related Work

A. Steam-specific popularity and success prediction

Several studies predict Steam game outcomes directly. De Luisa et al. [1] use a Bayesian hierarchical model to predict early player counts from price, size, supported languages, release date, and genre. A genre-based hierarchical version performs best. Its accuracy drops sharply beyond the second month after release. Hu et al. [2] compare seven models, including LightGBM and deep learning variants, to classify Steam games by positive-review rate. Their reported LightGBM accuracy differs between the paper's abstract and its results section.

Du [11] predicts whether a Steam game is currently on discount, using logistic regression and random forest. Random forest reaches 79.5 percent accuracy. This task is adjacent to success prediction rather than a direct measure of it. Ma et al. [9] merge Steam, Twitch, and Metacritic data for 1,121 games. They use gradient boosting and find that Steam-derived metrics are the strongest predictors of revenue. The merged sample is selection-biased toward titles already popular enough to be covered by Metacritic.

Lei et al. [13] and AlHemeiri [17] both study the link between Steam review sentiment and popularity. Both depend on review text and engagement telemetry collected after release. Wirawan and Kusuma [16] use time-lagged historical features from SteamDB to predict daily player counts for the 100 most-played Steam titles. Their result depends on games already having an established player base. It does not generalize to a title's pre-release stage.

Walter et al. [18] model demand for a single live-service title, Apex Legends, using a hybrid epidemic-style dynamical system rather than a supervised classifier. This differs enough from a supervised success prediction task to offer limited direct comparison. What remains unsolved in this group is a model that only uses information available before a game's release, on a dataset large enough to generalize past a handful of titles. It must also treat the outcome tiers as ordered, rather than as unrelated classes.

B. Success prediction outside the Steam ecosystem

A parallel line of work predicts commercial success for mobile games, movies, and crowdfunding campaigns. Several of these studies share methodology with the Steam-specific work. Shovo et al. [3] merge multiple video game sales datasets, including a general video game sales dataset and a smaller PC-specific sales dataset. They compare logistic regression against a random forest classifier to predict a binary hit-or-not-hit commercial success label, reaching about 85 percent and 84 percent accuracy respectively. This task predicts general post-launch commercial success from a broad multi-platform dataset, rather than Steam-specific pre-release metadata. Muhtasim and Hossen [4] predict Google Play mobile game downloads and ratings from pre-release metadata. They find that category, supported language count, and developer profile are the strongest predictors.

Kerim and Genç [12] study over 17,000 mobile games and build a composite success score from revenue, reputation, and release speed. Their model reaches roughly 70 percent classification accuracy using only metadata. They note that their causal interpretations may be biased. The study only finds correlations, not causal mechanisms. Mahmud et al. [7] predict movie box office revenue from pre-release metadata using an ordered revenue class scheme. The scheme scores predictions by how many classes away they fall from the true class, close to the ordinal approach used in this paper.

Zhou et al. [8] instead predict movie revenue with a multimodal deep network that learns features directly from movie posters. Zhang et al. [14] and Chao et al. [15] extend movie revenue prediction with sentiment mining and multimodal pretraining respectively. Both report gains from adding richer pre-release signal on top of basic metadata. Tian [6] and Elitzur et al. [20] predict Kickstarter campaign success using random forest and gradient boosting. Both studies find that machine learning models outperform standard logistic regression once complex, non-linear relationships between predictors are present.

Li et al. [10] and Aziz et al. [19] predict video game sales rather than success tiers. Both work from the VGChartz sales database rather than Steam. This makes this group adjacent to the present study rather than directly comparable. What remains unsolved in this group is that none of these studies work on the Steam platform specifically. Few of them evaluate their models with a metric built for ordered outcomes, even where the underlying target is naturally graded.

C. The ordinal structure gap

Across both groups, almost every reviewed study treats success prediction as either a regression problem on a continuous outcome or a classification problem on an unordered label set [1], [2], [4], [10], [12], [19]. Very few studies score their models with a metric built for ordered outcomes, such as quadratic weighted kappa [2], [7], [12], [16]. Prasad's unfinished proposal [5] plans to combine standard pre-release metadata with sentiment scores drawn from game plot text. It explicitly cautions that using owner or sales counts as a success measure, as prior work had done, is not fully reliable. The proposal was never carried out and produced no results. None of the reviewed papers, including this proposal, frame their target as an ordered set of tiers scored by how close a prediction lands to the true tier.

This paper addresses that remaining gap directly. It builds a four-tier ordinal target from Steam metadata and compares an ordinal regression reframing against standard classifiers, using metrics suited to ordered outcomes.

III. Methodology

A. Overview

The pipeline runs from a raw Steam metadata dump to a final evaluated model, shown in Fig. 1. The pipeline first repairs a known schema defect in the raw file. It next builds a four-tier ordinal target from estimated owner counts and applies a leakage guard that keeps only columns known before a game's release. It then cleans the data, engineers 54 features, splits the data with stratification, and scales it. Finally, it trains and evaluates a set of models.



Fig. 1. Data pipeline from the raw Steam metadata dump to model evaluation.

B. Target construction

The raw dataset reports game ownership as a range rather than an exact count, for example 20,000 to 50,000. Let l_i and u_i denote the lower and upper bound of the owner range reported for game i. The point estimate of ownership used in this paper is the midpoint of that range:

$$m_i = \frac{l_i + u_i}{2} \tag{1}$$

where m_i is the estimated owner count for game i. Games with $m_i = 0$ are removed. A zero-owner record most often indicates an unreleased or delisted entry, not a released product with no players. The target label y_i for game i is then assigned by four hand-set cut points on m_i:

$$y_i = \begin{cases} 0 \text{ (Niche)} & m_i \le 10{,}000 \\ 1 \text{ (Moderate)} & 10{,}000 < m_i \le 75{,}000 \\ 2 \text{ (High)} & 75{,}000 < m_i \le 350{,}000 \\ 3 \text{ (Hit)} & m_i > 350{,}000 \end{cases} \tag{2}$$

These cut points were chosen from the natural breaks in the owner bands, not from an optimization procedure. This choice is treated as a limitation in Section VI.

C. Leakage guard

A feature is only usable if it would be known to a developer before a game's release. Fifteen columns that only exist after release, including peak player counts, review counts, review scores, and total playtime, are removed. Five identifier or empty columns are also removed. Two additional columns, current discount level and downloadable content count, are removed for the same reason: both accumulate value only after a game has launched.

D. Feature engineering

After the leakage guard, 54 features are built across six groups. The first group is presence flags for website, support URL, and support email. The second group is date-derived features, including release year, month, day of week, a weekend flag, and days since release relative to a fixed reference date. The third group is platform and pricing features, including platform count, a multiplatform flag, a free-to-play flag, and price in US dollars.

The fourth group is content richness features, including screenshot count, About text word count, and achievement count. The fifth group is language features, including supported language count, audio language count, and an English-support flag. The sixth group is studio features, including self-publishing status, publisher presence, and a frequency count of how many games a developer or publisher has previously released. Genre and category values are encoded using counts plus a multi-hot indicator for the twelve most common labels in each field. This avoids a full one-hot expansion over the long tail of rare labels. Skewed continuous features are log-transformed before scaling. All continuous features are then standardized to zero mean and unit variance after the train/test split, using the statistics of the training partition only.

E. Ordinal regression formulation

The final model reframes the four-tier classification problem as an ordinal regression. A gradient boosting regressor f_θ is trained to predict a single continuous score $\hat{s}_i = f_\theta(x_i)$ for each game. The integer-coded tier $y_i \in \{0, 1, 2, 3\}$ is used as the regression target. Three cut points $c_1 < c_2 < c_3$ are then

chosen to convert the continuous score back into a discrete tier:

$$\hat{y}_i = \begin{cases} 0 & \hat{s}_i \le c_1 \\ 1 & c_1 < \hat{s}_i \le c_2 \\ 2 & c_2 < \hat{s}_i \le c_3 \\ 3 & \hat{s}_i > c_3 \end{cases} \tag{3}$$

The cut points are selected by a grid search over quantiles of the training set predictions. The search chooses the combination (c_1, c_2, c_3) that maximizes macro F1 on the training set, subject to $c_1 < c_2 < c_3$. This search is described in Algorithm 1.

Algorithm 1 Cut point search for the ordinal reframing

Train regressor f_θ on $(X_{\text{train}}, y_{\text{train}})$
$\hat{s}_{\text{train}} \leftarrow f_\theta(X_{\text{train}})$
$G \leftarrow$ 22 quantiles of $\hat{s}_{\text{train}}$ between the 50th and 98th percentile
best $\leftarrow (-1, \text{None})$
for each $(c_1, c_2, c_3) \in G \times G \times G$ with $c_1 < c_2 < c_3$ **do**
  $\hat{y}_{\text{train}} \leftarrow$ digitize $\hat{s}_{\text{train}}$ using (c_1, c_2, c_3)
  $s \leftarrow$ macro F1 of $\hat{y}_{\text{train}}$ against y_{train}
  if $s >$ best.$score$ **then**
    best $\leftarrow (s, (c_1, c_2, c_3))$
  end if
end for
return best.$cuts$

This approach differs from a standard multi-class classifier. It forces the model to place games on a single ordered axis. An error that places a Hit game in the High tier is treated by the underlying regression loss as smaller than an error that places the same game in the Niche tier. A standard multi-class classifier does not carry this property. It has no notion of which wrong classes are close to the correct one.

IV. Experimental Setup

A. Dataset

The dataset is the FronkonGames Steam Games Dataset, sourced from Kaggle and downloaded from a shared copy for this study. The raw file contains 125,855 rows and 40 columns. The published header row lists only 39 names, because the discount level and downloadable content count columns are merged under one header in the raw file. This is corrected with an explicit 40-column schema before parsing. After removing zero-owner records, 101,276 games remain. The tier distribution over these games is severely imbalanced, as shown in Fig. 2: Niche accounts for 74.8 percent of games, Moderate for 16.5 percent, High for 6.2 percent, and Hit for only 2.5 percent.

B. Evaluation metrics

Four metrics are used to evaluate every model. Accuracy is the share of predictions that exactly match the true tier. Balanced accuracy is the average of the per-class recall values.

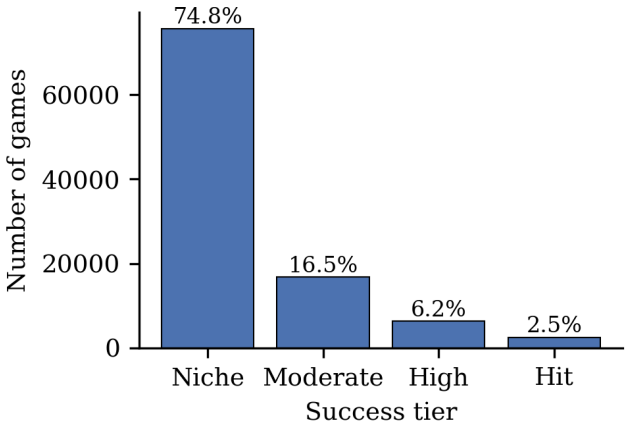


Fig. 2. Distribution of games across the four success tiers.

This prevents a model from scoring well by only predicting the majority tier.

Macro F1 is the unweighted average of the per-class F1 scores across all four tiers. Quadratic weighted kappa measures agreement between predicted and true tiers. It gives a larger penalty to errors that land far from the true tier than to errors that land close to it. It is the primary metric used in this paper to judge the ordinal quality of a model's mistakes. For the binary breakout extension in Section V-C, ROC AUC and PR AUC are also reported. For the leakage check, the drop in macro F1 after removing a feature is used directly as the importance measure.

C. Implementation details

All models are implemented in scikit-learn. LightGBM is used for the gradient boosting comparison model. A fixed random seed of 42 is used throughout, for the train/test split, the cross-validation folds, and every stochastic model. The data is split 80/20 with stratification on the tier label. This gives 81,020 training games and 20,256 test games, each with 54 features. No separate validation set is held out; five-fold stratified cross-validation on the training set is used instead for model selection and hyperparameter tuning.

The five baseline models are a most-frequent-class dummy classifier, logistic regression with balanced class weights and a maximum of 1,000 iterations, a decision tree with balanced class weights, a random forest with 200 trees and balanced class weights, and a histogram-based gradient boosting classifier at its library default settings. The gradient boosting model is then tuned using a randomized search of 25 candidate settings over learning rate, maximum boosting iterations, maximum leaf nodes, minimum samples per leaf, and L2 regularization strength, scored by five-fold cross-validated macro F1. The search selects a learning rate of 0.059, a maximum of 389 boosting iterations, 54 maximum leaf nodes, a minimum of 101 samples per leaf, and an L2 regularization strength of 0.177. Three sample weighting strategies are then compared using the tuned settings: no weighting, fully balanced

weighting, and a soft weighting equal to the square root of the balanced weights. The ordinal regression model uses the same tuned gradient boosting settings, but as a regressor, with cut points selected as described in Algorithm 1. The LightGBM comparison model uses 400 estimators, a learning rate of 0.05, 48 leaves, and balanced class weights.

V. Results and Discussion

A. Baseline comparison

Table I shows the five-fold cross-validated macro F1 and the held-out test metrics for the five baseline models. The dummy classifier always predicts the majority Niche tier. It reaches 74.75 percent accuracy but a macro F1 of only 0.2139 and a kappa of exactly 0. This confirms that accuracy alone is not a usable metric on this dataset.

Random forest reaches the highest raw accuracy among the trained models, at 78.68 percent. Its macro F1 of 0.4983 is below gradient boosting, because it favors the majority tier at the expense of the rarer ones. It is nonetheless the second-best baseline on macro F1, ahead of logistic regression, the decision tree, and the dummy classifier. Gradient boosting has the best cross-validated and test macro F1 among the five baselines, at 0.5189 and 0.5197 respectively. It is chosen as the model to tune further.

TABLE I
Baseline Model Comparison (Test Set)

Model	Acc.	Bal. Acc.	Macro F1	κ
Dummy (majority)	0.7475	0.2500	0.2139	0.0000
Decision Tree	0.7050	0.4527	0.4535	0.4776
Logistic Regression	0.6451	0.5623	0.4664	0.5500
Random Forest	0.7868	0.4544	0.4983	0.5358
Gradient Boosting	0.6761	0.5979	0.5197	0.5917

B. Effect of tuning, weighting, and the ordinal reframe

Table II compares the tuned gradient boosting model under three weighting strategies against the ordinal regression reframe and LightGBM. Full balanced weighting gives the worst macro F1 of the group, at 0.5241. Strong weighting raises recall on the rare tiers at a steep cost to precision. This cost is largest for the Hit tier, whose precision falls to 0.4031 under full balancing. The soft weighting strategy uses the square root of the balanced weights.

Soft weighting recovers most of this loss while keeping some of the recall gain, reaching a macro F1 of 0.5550. The ordinal regression reframe outperforms every classifier in the comparison, with a macro F1 of 0.5594 and a kappa of 0.6691, the highest of any model tested. Fig. 3 shows this comparison directly. The gain from the ordinal reframe over the best classifier, gradient boosting with soft weighting, is small in macro F1 but larger in kappa. This is consistent with the ordinal model producing errors that are closer to the true tier, rather than simply producing fewer errors overall.

TABLE II
Final Model Comparison (Test Set)

Model	Acc.	Bal. Acc.	Macro F1	κ
GB baseline	0.6761	0.5979	0.5197	0.5917
GB tuned (balanced)	0.6776	0.6024	0.5241	0.5972
LightGBM	0.6988	0.6012	0.5470	0.6183
GB tuned (soft)	0.7678	0.5624	0.5550	0.6656
Ordinal regression	**0.7539**	**0.5406**	**0.5594**	**0.6691**

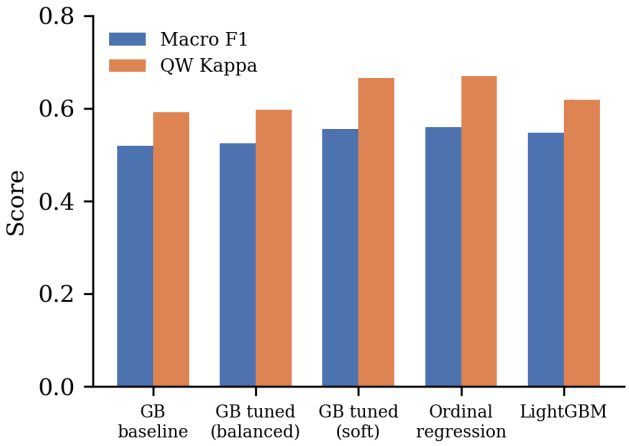


Fig. 3. Macro F1 and quadratic weighted kappa across the final model comparison.

C. Binary and three-tier extensions

Two simplified reframings of the target were also tested, to check whether the four-tier boundary itself is the main source of difficulty. A binary breakout model asks only whether a game rises above the Niche tier. It reaches a ROC AUC of 0.8802 and a PR AUC of 0.7452. It correctly identifies 76.8 percent of breakout games at a 0.5 decision threshold, with 91.1 percent precision on the Niche class and 56.8 percent precision on the Breakout class.

A three-tier model, formed by merging the High and Hit tiers, reaches a macro F1 of 0.6051 and a balanced accuracy of 0.6672, both higher than the four-tier result. This confirms that part of the difficulty in the main task comes from separating the two rarest tiers, High and Hit. Together, these two tiers make up only 8.7 percent of the dataset.

D. Feature importance and leakage check

Fig. 4 shows the permutation importance of the ten most influential features for the final ordinal model, measured on a 5,000-row sample of the test set. Two features dominate the ranking by a wide margin: release age in days, at 0.2624, and tag count, at 0.2509. Every other feature falls below 0.06. Both dominant features carry some risk of leaking post-release information into a pre-release prediction task. Tag count can grow after a game's community has formed opinions about it.

Release age is mechanically tied to how long a game has been available for players to discover.

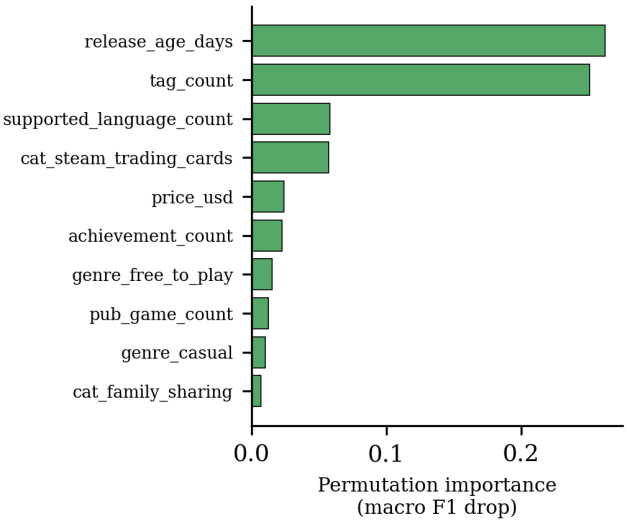


Fig. 4. Permutation importance of the top ten pre-release features for the ordinal regression model.

To check how much the model depends on these two features, both are removed. The ordinal model is retrained on the remaining 52 features. Macro F1 falls from 0.5594 to 0.5242, a drop of 0.0351. This is a real drop, but the model still retains most of its predictive power without the two dominant features. This suggests the ordinal model is not built entirely on leakage-prone signal, even though its single strongest feature is somewhat leakage-adjacent.

Store page richness features, such as About text word count, were expected to rank highly under the project's original hypothesis. None of them appear in the top ten. This suggests that the time and community signals carried by the two dominant features overlap with, and crowd out, the influence of effort signals visible on a game's store page.

VI. Limitations and Threats to Validity

Several limitations affect how far these results can be trusted. The success tiers are built from an owners range, not an exact sales figure, so the target carries some noise from the width of each Steam-reported band. The four tier boundaries at 10,000, 75,000, and 350,000 owners were set by hand from the natural breaks in the data, not through a search over possible cut points. A different set of boundaries would shift every downstream result. Every experiment in this paper uses a single random seed, so no result here carries a reported spread across repeated runs. This limits how confidently the exact metric values can be generalized.

Only a single train/test split is used. Five-fold cross-validation on the training partition stands in for a held-out validation set, rather than a fully separate three-way split. The two most influential features, release age and tag count, are both somewhat leakage-adjacent. The leakage check in Section V-D shows the model does not collapse without them, but they still dominate the full model's decisions.

LightGBM was compared using fixed hyperparameter settings, not the same tuning procedure applied to the gradient boosting model. This means the comparison between the two boosting libraries is not on fully equal footing. Class imbalance remains severe even after tier merging. The Hit tier, at 2.5 percent of the dataset, remains the hardest tier to predict with high precision across every model tested.

VII. Conclusion and Future Work

This paper frames Steam game success-tier prediction as an ordinal regression problem built entirely from pre-release metadata. This framing outperforms five standard classifiers, a tuned and reweighted gradient boosting model, and a LightGBM comparison model, on a large, cleaned Steam dataset. The ordinal regression model reaches a macro F1 of 0.5594 and a quadratic weighted kappa of 0.6691. A leakage check confirms the result is not built entirely on its two most dominant, leakage-adjacent features.

Future work should search over tier cut points, rather than fixing them by hand. It should also repeat the experiment across multiple random seeds, to report a spread on every metric. LightGBM should be tuned with the same search procedure used for the gradient boosting model. Future work should also test resampling methods aimed specifically at raising recall on the Hit tier, without the steep precision cost seen under full balanced weighting in this study.

Acknowledgment

The authors thank the course instructor for the guidance provided on structuring this paper from the project's earlier phases.

References

[1] A. De Luisa, J. Hartman, D. Nabergoj, S. Pahor, M. Rus, B. Stevanoski, J. Demšar, and E. Štrumbelj, "Predicting the popularity of games on Steam," Elektrotehniški Vestnik, vol. 88, no. 4, pp. 151–162, 2021.

[2] W. Hu, Y. Wang, and R. Xia, "Machine learning-based Steam platform game's popularity analysis," in Proc. 1st Int. Conf. Data Sci. Eng. (ICDSE 2024), Chengdu, China, 2024, pp. 582–588.

[3] S. A. R. Shovo et al., "Predictive analysis on commercial success of game," ResearchGate preprint, Apr. 2023.

[4] M. Muhtasim and M. S. Hossen, "Data analysis and success prediction of mobile games before publishing on Google Play Store," Int. J. Educ. Manag. Eng. (IJEME), vol. 14, no. 3, pp. 13–21, Jun. 2024.

[5] A. Prasad, "Estimating video game success using machine learning," research proposal, Aug. 2019.

[6] J. Tian, "Do you want to foresee your future? The best model predicting the success of Kickstarter campaigns," in Proc. 2021 13th Int. Conf. Mach. Learn. Comput. (ICMLC 2021), Feb. 2021.

[7] Q. I. Mahmud, N. Z. Shuchi, F. M. Tawsif, A. Mohaimen, and A. Tasnim, "A machine learning approach to predict movie revenue based on pre-released movie metadata," J. Comput. Sci., vol. 16, no. 6, pp. 749–767, 2020.

[8] Y. Zhou, L. Zhang, and Z. Yi, "Predicting movie box-office revenues using deep neural networks," Neural Comput. Appl., vol. 31, pp. 1855–1865, 2019.

[9] R. Ma et al., "Unveiling success drivers in gaming: A machine learning study across Steam, Twitch, and Metacritic," Int. J. Comput. Games Technol., vol. 2025, 2025.

[10] J. Li, Y. Zheng, H. Hu, J. Lu, and C. Zhan, "Predicting video game sales based on machine learning and hybrid feature selection method," in Proc. 2021 16th Int. Conf. Intell. Syst. Knowl. Eng. (ISKE), 2021, pp. 497–502.
[11] L. Du, "Steam game discount prediction using machine learning methods," in Proc. 2021 3rd Int. Conf. Mach. Learn. Big Data Bus. Intell. (MLBDBI), Taiyuan, China, 2021, pp. 149–152.
[12] A. Kerim and B. Genç, "Mobile games success and failure: Mining the hidden factors," Neural Comput. Appl., vol. 37, no. 2, pp. 543–557, 2025.
[13] M. Lei, J. Zhang, and Z. Zhan, "Leveraging sentiment analysis of Steam player reviews to predict game popularity," in Proc. 2025 8th Artif. Intell. Cloud Comput. Conf. (AICCC 2025), 2025.
[14] Z. Zhang, Y. Meng, and D. Xiao, "Prediction techniques of movie box office using neural networks and emotional mining," Sci. Rep., vol. 14, art. 21209, 2024.
[15] Q. Chao, E. Kim, and B. Li, "Copycat vs. original: Multi-modal pretraining and variable importance in box-office prediction," arXiv:2509.15277, Sep. 2025.
[16] G. H. Wirawan and G. P. Kusuma, "Predicting the number of video game players on the Steam platform using machine learning and time lagged features," Int. J. Adv. Comput. Sci. Appl. (IJACSA), vol. 15, no. 12, 2024.
[17] K. AlHemeiri, "Integrating sentiment analysis in predictive models: A comparative study on game popularity on Steam," CMC Senior Theses, no. 3723, Claremont McKenna College, 2025.
[18] I. Walter, J. H. Panchal, and P. E. Paré, "Hybrid SIS dynamics for demand modeling of frequently updated products," arXiv:2506.01866, 2025.
[19] A. Aziz, S. Ismail, M. Fakri Othman, and A. Mustapha, "Empirical analysis on sales of video games: A data mining approach," J. Phys. Conf. Ser., vol. 1049, art. 012086, 2018.
[20] R. Elitzur, N. Katz, P. Muttath, and D. Soberman, "The power of machine learning methods to predict crowdfunding success: Accounting for complex relationships efficiently," J. Bus. Venturing Design, vol. 3, art. 100022, 2024.